4 purrr

In this section we step beyond simple vectorised operations and look at functions that produce lists and objects. We then see how these can be incorporated into a `tidyverse` / `dplyr` framework with relative ease.

By the end of this section, you should be able to:



- Use the `map()` function and it's variations to transform an input by applying a function **to each element of an object** and returning an object of the **same length as the input**.
- Use the `map()` function instead of loops which decrease code readability and are known to be slow in R when not used properly.
- Use the `tidyr` functions `nest()` and `unnest()`.
- Use `nest()` and `unnest()` in conjunction with `map()` to apply a function to a list of data frames.
- Understand why the `map()` function is advantageous.

4.1 Looping with `map()`

The `map()` function lives in the `purrr` package and is the `tidyverse` equivalent to the base R function `lapply()`.

Here's a trivial example of `map()` using a list of vectors.

```
# Load the packages
library(purrr)

# Create a list of vectors
simple_list <- list(
  a = 1:10,
  b = rnorm(6)
)
simple_list
```

```
$a
 [1]  1  2  3  4  5  6  7  8  9 10
```

```
$b
[1] -1.0651865  0.7084692 -1.0711152 -1.1665183 -1.5166125 -0.7212341
```

Now that we have 2 vectors in a list, how do we loop through both of them to find each vector's mean?

We can use the `map()` function to loop through the list of vectors and produce a list of means. The function requires two inputs:

1. A list
2. A function to call on each element of the list

```
# Calculate the mean of each vector
map(simple_list, mean)
```

```
$a
[1] 5.5

$b
[1] -0.8053662
```

Alternatively we can use the `lapply()` function which is the base R equivalent of `map()`

```
# Calculate the mean of each vector
lapply(simple_list, mean)
```

```
$a
[1] 5.5

$b
[1] -0.8053662
```

Both the `lapply()` and `map()` functions can be used interchangeably. They have the **same syntax**, **same function**, and produce the **same output**. Although the apply family functions are slightly faster it is better programming practice to use the map family functions as they use **simple syntax and are expressive**. You can have an in depth read about the advantages and disadvantages of `lapply()` vs `map()` on stack overflow.

In the previous example we used vectors in a list but we can, of course, use data frames. In this example we use `map()` to look at the `head()` (first 6 rows) of each data frame.

```
vs <- read_sas("./data/vs.sas7bdat")
dm <- read_sas("./data/dm.sas7bdat")

pop <- vs %>%
  left_join(dm, by = "USUBJID")

# Apply map directly to the dataframe
map(pop, head)
```

```
$USUBJID
[1] "STD123456:000001" "STD123456:000002" "STD123456:000003" "STD123456:000004"
[5] "STD123456:000005" "STD123456:000006"

$HEIGHT
[1] 165 179 182 166 169 164

$WEIGHT
[1]  88.0 100.0  81.0  91.0  73.6  66.0

$AGE
[1] 32 28 55 35 30 22

$SEX
[1] "F" "M" "M" "F" "F" "F"

$COUNTRY
[1] "UK"  "FRA" "USA" "GER" "IRE" "GER"

$ARM
[1] "Comparator" "Comparator" "Comparator" "Comparator" "Comparator"
[6] "Comparator"
```

In the following example we manually split the pop data into males and females, put them in a list and then apply the mapping function to look at the `head()` (first 6 rows) of each data frame.

```
# Create a list of dataframes, grouped by sex
df_list <- list(
  a = filter(pop, SEX == "F"),
  b = filter(pop, SEX == "M")
)
df_list
```

```
$a
# A tibble: 20 × 7
   USUBJID          HEIGHT WEIGHT   AGE SEX   COUNTRY ARM
   <chr>             <dbl>  <dbl> <dbl> <chr> <chr>   <chr>
 1 STD123456:000001    165   88      32 F     UK      Comparator
 2 STD123456:000004    166   91      35 F     GER     Comparator
 3 STD123456:000005    169   73.6    30 F     IRE     Comparator
 4 STD123456:000006    164   66      22 F     GER     Comparator
 5 STD123456:000007    157  107      59 F     USA     Comparator
 6 STD123456:000009    155   54.1    60 F     USA     GSK
 7 STD123456:000011    173   78      66 F     USA     GSK
 8 STD123456:000013    156   64      51 F     IRE     GSK
 9 STD123456:000014    161   54      48 F     USA     Comparator
10 STD123456:000015    164  102.     59 F     FRA     GSK
11 STD123456:000016    157   64.5    59 F     USA     Comparator
12 STD123456:000017    170   93.6    28 F     FRA     GSK
13 STD123456:000018    169  106.     69 F     IRE     Comparator
14 STD123456:000019    153   65      48 F     USA     GSK
```

```
15 STD123456:000023    166   79      28 F     IRE     Comparator
16 STD123456:000024    151   58      50 F     FRA     Comparator
17 STD123456:000025    159   65      48 F     USA     GSK
18 STD123456:000026    163  116      71 F     FRA     GSK
19 STD123456:000027    170   97      67 F     IRE     Comparator
20 STD123456:000029    151   84      68 F     UK      Comparator

$b
# A tibble: 10 × 7
   USUBJID          HEIGHT WEIGHT   AGE SEX   COUNTRY ARM
   <chr>             <dbl>  <dbl> <dbl> <chr> <chr>   <chr>
 1 STD123456:000002    179    100    28 M     FRA     Comparator
 2 STD123456:000003    182     81    55 M     USA     Comparator
 3 STD123456:000008    189    101    53 M     GER     GSK
 4 STD123456:000010    171     88    48 M     USA     Comparator
 5 STD123456:000012    181     81    26 M     FRA     GSK
 6 STD123456:000020    175     82    64 M     FRA     Comparator
 7 STD123456:000021    195     86    55 M     GER     Comparator
 8 STD123456:000022    173     73    46 M     GER     GSK
 9 STD123456:000028    180     95    67 M     IRE     GSK
10 STD123456:000030    179    107    71 M     USA     Comparator
```

```r
# Equivalently
# df_list <- split(pop, pull(pop, SEX))

# First six rows of each element of the list
map(df_list, head)
```

```
$a
# A tibble: 6 × 7
  USUBJID          HEIGHT WEIGHT   AGE SEX   COUNTRY ARM
  <chr>             <dbl>  <dbl> <dbl> <chr> <chr>   <chr>
1 STD123456:000001    165   88      32 F     UK      Comparator
2 STD123456:000004    166   91      35 F     GER     Comparator
3 STD123456:000005    169   73.6    30 F     IRE     Comparator
4 STD123456:000006    164   66      22 F     GER     Comparator
5 STD123456:000007    157  107      59 F     USA     Comparator
6 STD123456:000009    155   54.1    60 F     USA     GSK

$b
# A tibble: 6 × 7
  USUBJID          HEIGHT WEIGHT   AGE SEX   COUNTRY ARM
  <chr>             <dbl>  <dbl> <dbl> <chr> <chr>   <chr>
1 STD123456:000002    179    100    28 M     FRA     Comparator
2 STD123456:000003    182     81    55 M     USA     Comparator
3 STD123456:000008    189    101    53 M     GER     GSK
4 STD123456:000010    171     88    48 M     USA     Comparator
5 STD123456:000012    181     81    26 M     FRA     GSK
6 STD123456:000020    175     82    64 M     FRA     Comparator
```

`map()` always returns a list

4.1.1 Missing Data

What would happen if you had missing values in your list and you applied the mapping function?

```
# List containing vector
simple_list2 <- list(a = c(1, 3, 4, 5, NA, 6))

# Finding the mean of the vector
map(simple_list2, mean)
```

```
$a
[1] NA
```

```
# Finding the sum of the vector
map(simple_list2, sum)
```

```
$a
[1] NA
```

Remember to remove NA's if you are doing any calculations in map. You can use the `na.rm=TRUE` as an extra argument **within the `map()` function** to do this.

```
# List containing vector
simple_list2 <- list(a = c(1, 3, 4, 5, NA, 6))

# Finding the mean of the vector after removing NA's
map(simple_list2, mean, na.rm=TRUE)
```

```
$a
[1] 3.8
```

4.1.2 Passing Additional Arguments to `map()`

The call to map follows this format: `map(.x, .f, ...)` .x is our list, .f is the function we want to apply to our list and the ellipses represents any additional arguments we might want to pass to `map()`

Additional arguments to the function that we call each time (e.g. `head()`) can be passed **directly to the `map()` function**. For example, the `head()` function lets us specify the number of rows to display, `n`. So to call the `head()` function on each element of our list, returning 2 rows each time we would write the following:

```
# First two rows of each element of the list
map(df_list, head, n = 2)
```

```
$a
# A tibble: 2 × 7
  USUBJID          HEIGHT WEIGHT   AGE SEX   COUNTRY ARM
  <chr>             <dbl>  <dbl> <dbl> <chr> <chr>   <chr>
```

```
1 STD123456:000001    165     88     32 F     UK      Comparator
2 STD123456:000004    166     91     35 F     GER     Comparator

$b
# A tibble: 2 × 7
  USUBJID          HEIGHT WEIGHT   AGE SEX   COUNTRY ARM
  <chr>             <dbl>  <dbl> <dbl> <chr> <chr>   <chr>
1 STD123456:000002    179    100    28 M     FRA     Comparator
2 STD123456:000003    182     81    55 M     USA     Comparator
```

`map()` uses the ellipsis to pass any additional arguments that we supply to `map()`, directly through to the `head()` function for us.

`map(df_list, head(2))` would not work!

4.1.3 Passing Custom Functions to `map()`

There are two ways to provide custom functions to `map()`:

1. Define the custom function and apply it as we do in the example below.
2. Use the special `~` to define small functions

Suppose we wish to fit a specific linear model to each data frame in our list. We are using the data frame `df_list` in this example. The first method requires us to **define the model as a function**.

```
# Define function
model_weight <- function(data) {
  lm(WEIGHT ~ HEIGHT, data = data)
}
# Apply function to list data
map(df_list, model_weight)
```

```
$a

Call:
lm(formula = WEIGHT ~ HEIGHT, data = data)

Coefficients:
(Intercept)       HEIGHT
    -125.44         1.27


$b

Call:
lm(formula = WEIGHT ~ HEIGHT, data = data)

Coefficients:
(Intercept)       HEIGHT
    25.8113       0.3525
```

In the example below we have made use of **anonymous functions**.

An anonymous function (also known as a lambda expression) is a function definition that is not bound to an identifier. It is a function that is created and used, but **never assigned to an object**.

The `purrr` package allows us to define anonymous functions when using the apply family functions:

In the example demonstrated above we have constructed a function with formal arguments and a body. However, when we define an anonymous function like in the example below, this is not required.

Here, we **use `~` to define the custom function** and `.` to specify what to do with each element of the list. So the following code effectively says: **"For each element (data frame) of `df_list`, fit a linear model of `WEIGHT` against `HEIGHT`".**

```
map(df_list, ~ lm(WEIGHT ~ HEIGHT, data = .))
```

```
$a

Call:
lm(formula = WEIGHT ~ HEIGHT, data = .)

Coefficients:
(Intercept)       HEIGHT
    -125.44         1.27


$b

Call:
lm(formula = WEIGHT ~ HEIGHT, data = .)

Coefficients:
(Intercept)       HEIGHT
    25.8113       0.3525
```

```
# Alternatively
map(df_list, function(data) lm(WEIGHT ~ HEIGHT, data = data))
```

```
$a

Call:
lm(formula = WEIGHT ~ HEIGHT, data = data)

Coefficients:
(Intercept)       HEIGHT
    -125.44         1.27


$b
```

```
Call:
lm(formula = WEIGHT ~ HEIGHT, data = data)

Coefficients:
(Intercept)       HEIGHT
    25.8113       0.3525
```

4.2 The map family

Just like apply family(`apply()` , `lapply()` , `tapply()` , `vapply()` , etc) functions in base R `purrr` package provides several version of the map function that can solve similar problems.

Here we will look into the following four functions: `map2()` , `pmap()` , `invoke_map()` and `imap()` .

4.2.1 Apply a function to a pair of elements from two different lists or vectors using `map2()`

To use the `map2()` function you must make sure both lists/vectors are of the same length.

Suppose we wanted to calculate x to the power of y and output a list of the numbers.

```
# Create two vectors of the same length
x <- c(2, 4, 5, 6)
y <- c(2, 3, 4, 5)
# Writing a function that creates x to the power of y
to_Power <- function(x, y) {
  return(x**y)
}
map2(x, y, to_Power)
```

```
[[1]]
[1] 4

[[2]]
[1] 64

[[3]]
[1] 625

[[4]]
[1] 7776
```

4.2.2 Map a function over multiple inputs simultaneously using `pmap()`

In this example we use a subset of the `mtcars` dataset. Then use `pmap()` to sum the columns on each row.

```
# Subsetting the mtcars dataset, rows 1-5 and columns mpg, hp and disp
mtcars_sub <- mtcars %>%
  slice(1:3) %>%
  select(mpg, hp, disp)
mtcars_sub
```

```
              mpg  hp disp
Mazda RX4     21.0 110  160
Mazda RX4 Wag 21.0 110  160
Datsun 710    22.8  93  108
```

```
# Sum the columns on every row of the mtcars_sub dataset
pmap(mtcars_sub, sum)
```

```
[[1]]
[1] 291

[[2]]
[1] 291

[[3]]
[1] 223.8
```

4.2.3 Invoke a list of functions on lists of arguments using `invoke_map()`

The `runif()` function allows drawing n random observations from a uniform distribution. The `rnorm()` function allows drawing n random observations from a normal distribution.

```
runif(n = 10)
```

```
 [1] 0.2694873 0.8254773 0.8608897 0.7605701 0.7484065 0.5136650 0.7765777
 [8] 0.4312298 0.5493351 0.1866868
```

```
rnorm(n = 5)
```

```
[1]  0.4918736 -0.2184535 -1.5003477 -2.1650500 -0.4660815
```

Invoke a function with a list of arguments.

```
invoke_map(runif, list(n = 10))
```

```
Warning: `invoke_map()` was deprecated in purrr 1.0.0.
ℹ Please use map() + exec() instead.

[[1]]
 [1] 0.38110841 0.07454009 0.21705058 0.62813592 0.21381207 0.65804930
 [7] 0.29805214 0.53515034 0.43189653 0.36289255
```

Invoke a list of functions, to a list of lists each with different arguments

```
invoke_map(list(runif, rnorm), list(list(n = 10), list(n = 5)))
```

```
[[1]]
 [1] 0.31093155 0.81309453 0.04906477 0.71065636 0.41881286 0.49438619
 [7] 0.12087391 0.16005516 0.14588660 0.91534866

[[2]]
[1] -0.85608118  0.70952187 -1.97014628  0.15243954  0.04423521
```

```
invoke_map(list(sum, mean, sum), list(list(5, 7, 8, 9), list(10, 12), list(8, 9)))
```

```
[[1]]
[1] 29

[[2]]
[1] 10

[[3]]
[1] 17
```

4.2.4 Compute a function on both the value and the position of an element using `imap()`

`imap()` is a function that can be generally thought of as an indexed map. The function uses 2 arguments.

1. The value of element
2. The position or name of the element

In this example we have a vector of 10 random numbers. Using `imap()` we output the sum of each number with its index. y is the index of each element in the list, x is the value of each element in the list.

```
imap(list(5, 10, 8, 12), ~ paste0(.x, ": ", .y))
```

```
[[1]]
[1] "5: 1"

[[2]]
[1] "10: 2"

[[3]]
[1] "8: 3"

[[4]]
[1] "12: 4"
```

```
imap(list(5, 10, 8, 12), ~ .x + .y)
```

```
[[1]]
[1] 6

[[2]]
[1] 12

[[3]]
[1] 11

[[4]]
[1] 16
```

Writing `imap(x, ~ .x + .y)` is the same as `map2(x, y, ~ .x + .y)`.

Here is another example of `imap()` using the `simple_list` we created before. In this case we have **named elements** therefore the index refers to the name instead of the integer. y is the named element, x is the value of the named element.

```
simple_list
```

```
$a
 [1]  1  2  3  4  5  6  7  8  9 10

$b
[1] -1.0651865  0.7084692 -1.0711152 -1.1665183 -1.5166125 -0.7212341
```

```
imap(simple_list, ~ paste0(.y, ": ", .x))
```

```
$a
 [1] "a: 1"  "a: 2"  "a: 3"  "a: 4"  "a: 5"  "a: 6"  "a: 7"  "a: 8"  "a: 9"
[10] "a: 10"

$b
[1] "b: -1.06518652235458"  "b: 0.708469217404014"  "b: -1.0711151715873"
[4] "b: -1.16651825965253"  "b: -1.51661247860878"  "b: -0.721234069914737"
```

4.2.5 Specifying the structure of your output.

All the functions we have looked at so far return a list. We must use a **suffixed version** to return the results as a specific type of flat vector e.g. `map_dbl()`, `imap_chr()` etc.

Function	Returns
map	list
map_chr	character vector
map_dbl	double(numeric) vector
map_dfc	data frame(column bind)
map_dfr	data frame(row bind)

Function	Returns
map_int	integer vector
map_lgl	logical vector
walk	tiggers side effects, returns the input invisibly

For example, if we want a vector instead of a list we can use the `_dbl` suffix. This returns a vector of type double.

```
# Create a list of vectors
simple_list <- list(
  a = 1:10,
  b = rnorm(6)
)
# Calculate the mean of each vector and output a vector
map_dbl(simple_list, mean)
```

```
        a          b
5.5000000 -0.3425951
```

If we want to return a character vector instead of a list we can use the suffix `_chr`.

```
# Invoke function to two lists of 10 random numbers
imap_chr(sample(10), ~ paste0(.y, ": ", .x))
```

```
 [1] "1: 7"  "2: 10" "3: 3"  "4: 8"  "5: 9"  "6: 1"  "7: 4"  "8: 2"  "9: 6"
[10] "10: 5"
```

4.3 Nested Data

In `map()`, we now have a tool that enables us to loop through and **perform actions on lots of data frames**. To make best use of this within the context of tibbles (specifically grouped tibbles), we will make use of the concept of ***nesting*** and the use of functions `nest()` and `unnest()` from the **`tidyr`** package.

Nesting creates a list-column of data frames. Unnesting flattens it back out into regular columns. Nesting is implicitly a summarising operation: you get one row for each group defined by the non-nested columns.

The `nest` function simply creates a **list of data frames from a tibble, preserving any grouping**. If there is no grouping, we provide the `data = everything()` option and it simply condenses down (nests) the entire data frame into a single cell within a column called **`data`**. The data column is, in fact, a list containing one element, a data frame (the original data).

We use a nested dataframe to:

1. Preserve relationships between observations and subsets of data.
2. Manipulate **many sub-tables at once** with **`purrr`** functions **map(), map2(), or pmap().**

A nested data frame is a data frame where one (or more) columns is a list of data frames. You can create simple nested data frames by hand:

```
df1 <- tibble(
  g = c(1, 2, 3),
  data = list(
    tibble(x = 1, y = 2),
    tibble(x = 4:5, y = 6:7),
    tibble(x = 10)
  )
)

df1
```

```
# A tibble: 3 × 2
      g data
  <dbl> <list>
1     1 <tibble [1 × 2]>
2     2 <tibble [2 × 2]>
3     3 <tibble [1 × 1]>
```

But more commonly you will create them with `nest()` along with the `group_by()` function: If our data are grouped by one or more columns, then `nest()` simply condenses down the remaining columns into a list of data frames.

```
df2 <- tribble(
  ~g, ~x, ~y,
  1, 1, 2,
  2, 4, 6,
  2, 5, 7,
  3, 10, NA
)

df2
```

```
# A tibble: 4 × 3
      g     x     y
  <dbl> <dbl> <dbl>
1     1     1     2
2     2     4     6
3     2     5     7
4     3    10    NA
```

```
df2 %>%
  group_by(g) %>%
  nest()
```

```
# A tibble: 3 × 2
# Groups:   g [3]
      g data
  <dbl> <list>
1     1 <tibble [1 × 2]>
2     2 <tibble [2 × 2]>
3     3 <tibble [1 × 2]>
```

Using our `pop` example:

```
# Load packages
library(tidyr)
pop
```

```
# A tibble: 30 × 7
   USUBJID          HEIGHT WEIGHT   AGE SEX   COUNTRY ARM
   <chr>             <dbl>  <dbl> <dbl> <chr> <chr>   <chr>
 1 STD123456:000001    165   88      32 F     UK      Comparator
 2 STD123456:000002    179  100      28 M     FRA     Comparator
 3 STD123456:000003    182   81      55 M     USA     Comparator
 4 STD123456:000004    166   91      35 F     GER     Comparator
 5 STD123456:000005    169   73.6    30 F     IRE     Comparator
 6 STD123456:000006    164   66      22 F     GER     Comparator
 7 STD123456:000007    157  107      59 F     USA     Comparator
 8 STD123456:000008    189  101      53 M     GER     GSK
 9 STD123456:000009    155   54.1    60 F     USA     GSK
10 STD123456:000010    171   88      48 M     USA     Comparator
# ℹ 20 more rows
```

```
pop %>%
  nest(data = everything())
```

```
# A tibble: 1 × 1
  data
  <list>
1 <tibble [30 × 7]>
```

```
nest_example <- pop %>%
  group_by(SEX) %>%
  nest()
nest_example
```

```
# A tibble: 2 × 2
# Groups:   SEX [2]
  SEX   data
  <chr> <list>
1 F     <tibble [20 × 6]>
2 M     <tibble [10 × 6]>
```

An alternative approach would be specify the columns you want grouped within the `nest()` function itself.

```
pop %>%
  nest(data = c(USUBJID, HEIGHT, WEIGHT, AGE, COUNTRY, ARM))
```

```
# A tibble: 2 × 2
  SEX   data
  <chr> <list>
1 F     <tibble [20 × 6]>
2 M     <tibble [10 × 6]>
```

Variables listed inside `nest()` specify which variables should be nested inside, whereas variables listed inside `group_by()` describe which variables should be kept outside.

The opposite of `nest()` is `unnest()`. You give it the name of a list-column containing data frames, and it **row-binds** the data frames together, repeating the outer columns the right number of times to line up.

```
unnest_example <- unnest(nest_example)
```

```
Warning: `cols` is now required when using `unnest()`.
ℹ Please use `cols = c(data)`.
```

```
unnest_example
```

```
# A tibble: 30 × 7
# Groups:   SEX [2]
   SEX   USUBJID          HEIGHT WEIGHT   AGE COUNTRY ARM
   <chr> <chr>             <dbl>  <dbl> <dbl> <chr>   <chr>
 1 F     STD123456:000001    165   88      32 UK      Comparator
 2 F     STD123456:000004    166   91      35 GER     Comparator
 3 F     STD123456:000005    169   73.6    30 IRE     Comparator
 4 F     STD123456:000006    164   66      22 GER     Comparator
 5 F     STD123456:000007    157  107      59 USA     Comparator
 6 F     STD123456:000009    155   54.1    60 USA     GSK
 7 F     STD123456:000011    173   78      66 USA     GSK
 8 F     STD123456:000013    156   64      51 IRE     GSK
 9 F     STD123456:000014    161   54      48 USA     Comparator
10 F     STD123456:000015    164  102.     59 FRA     GSK
# ℹ 20 more rows
```

4.4 Using `map()` with nested data

For large, simulated data frames, particular those with data at the subject level, the `nest()` function can be a useful tool for looking at the overall structure of our data. However, the main advantage of `nest()` is that it creates a list of data frames, **within a tibble**, to which we can fit models. Nesting is a great fit for problems where you want one of something for each group.

In the following example we use `mutate()` to create a new column called `model`. This is generated by fitting a linear model to each data frame nested within the `data` column.

```
pop
```

```
# A tibble: 30 × 7
   USUBJID          HEIGHT WEIGHT   AGE SEX   COUNTRY ARM
   <chr>             <dbl>  <dbl> <dbl> <chr> <chr>   <chr>
 1 STD123456:000001    165   88      32 F     UK      Comparator
 2 STD123456:000002    179  100      28 M     FRA     Comparator
 3 STD123456:000003    182   81      55 M     USA     Comparator
 4 STD123456:000004    166   91      35 F     GER     Comparator
 5 STD123456:000005    169   73.6    30 F     IRE     Comparator
 6 STD123456:000006    164   66      22 F     GER     Comparator
 7 STD123456:000007    157  107      59 F     USA     Comparator
 8 STD123456:000008    189  101      53 M     GER     GSK
 9 STD123456:000009    155   54.1    60 F     USA     GSK
10 STD123456:000010    171   88      48 M     USA     Comparator
# ℹ 20 more rows
```

```
pop_models <- pop %>%
  group_by(SEX) %>%
  nest() %>%
  mutate(model = map(data, ~ lm(WEIGHT ~ HEIGHT, data = .)))
pop_models
```

```
# A tibble: 2 × 3
# Groups:   SEX [2]
  SEX   data              model
  <chr> <list>            <list>
1 F     <tibble [20 × 6]> <lm>
2 M     <tibble [10 × 6]> <lm>
```

Extending this example we might choose to post-process the model, **extracting a data frame of key information** using the `tidy()` function. The `tidy()` function constructs a tibble that summarizes the model's statistical findings.

```
# Load the package
library(broom)

pop_models <- pop %>%
  group_by(SEX) %>%
  nest() %>%
  mutate(
    model = map(data, ~ lm(WEIGHT ~ HEIGHT, data = .)),
    key_info = map(model, tidy)
  )
pop_models
```

```
# A tibble: 2 × 4
# Groups:   SEX [2]
  SEX   data              model  key_info
```

```
  <chr> <list>           <list> <list>
1 F     <tibble [20 × 6]> <lm>   <tibble [2 × 5]>
2 M     <tibble [10 × 6]> <lm>   <tibble [2 × 5]>
```

Having fitted a number of models and extracted the key parameter information, the next step is to turn this into something readable. To do so we use `unnest()`.
With `unnest()`, we must be specific about which of the nested columns it is that we wish to unnest. From our previous example the **`key_info`** column will be most useful to us.

```
pop_models %>%
  unnest(key_info)
```

```
# A tibble: 4 × 8
# Groups:   SEX [2]
  SEX   data              model  term       estimate std.error statistic p.value
  <chr> <list>            <list> <chr>         <dbl>     <dbl>     <dbl>   <dbl>
1 F     <tibble [20 × 6]> <lm>   (Intercep… -125.       96.2      -1.30   0.209
2 F     <tibble [20 × 6]> <lm>   HEIGHT        1.27      0.593     2.14   0.0463
3 M     <tibble [10 × 6]> <lm>   (Intercep…   25.8      93.9       0.275  0.790
4 M     <tibble [10 × 6]> <lm>   HEIGHT        0.352     0.520     0.678  0.517
```

It is only possible to unnest list of vectors and data frames. It is not possible to unnest a list of lists. However, we could use `pull()` to extract the column and see it in its raw form.

4.4.1 Speed Advantage

The `map()` function is not always required. Many of the operations we might use it for are also possible using a `group_by()` operation, `while` loops or the `lapply()` function. However, `map()` is very fast, and a useful alternative.

In our final example we could have used a complex `while` loop to generate the same data however these are very slow in R as we are unable to predetermine the dimensions of the output structure. If we used a `group_by()` approach instead, we would have had to call the loop on hundreds of thousands of 'by' groups which also would have slowed it down.

4.5 EXERCISE

1. Using the iris dataset from **`tidyverse`** library, capitalize the "Species" variable in the dataset using a map function. ***HINT: Use the `map_at()` function***
2. Combine this list of words to make a sentence: `list=("I", "know", "how", "to", "use", "the", "mapping", "function")` ***HINT: Use the `invoke_map()` function***
3. map(1:3, ~ runif(2)) is a useful pattern for generating random numbers, but map(1:3, runif(2)) is not. Why not? Can you explain why it returns the result that it does?
4. Write a function to add two numbers x and y together. Create a vector x, y must be a randomly generated value. Use the function `map_dbl` to apply the function to x and y and produce a double

numeric list of values.

There is a subtle difference between placing extra arguments inside an anonymous function compared with passing them to `map()`. Putting them in an anonymous function means that they will be evaluated every time f() is executed, not just once when you call `map()`.

5. Use the iris dataset below to nest columns that have names beginning with “Petal” or “Sepal” and name the new nested columns petal and sepal respectively. Do this by specifying variables in the `nest()` function. ***HINT: Use the `starts_with` function from the `tidyr` package***

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa